

Lecture 16 Notes

Opening: Mixed Feelings About Coverage

The instructor opened the lecture reflectively, saying that sometimes it feels like the course has not covered much, and other times it feels like a great deal has been covered — “I have mixed feelings.” The hope expressed is that students feel they have gone beyond basic undergraduate linear algebra and have picked up concepts that are genuinely useful, and that the motivations for these subjects will become even clearer as later material builds upon them.

This lecture is primarily a **mid-course review** (the midterm is approaching) followed by a detailed treatment of **linear time-invariant (LTI) systems, circulant matrices, the DFT, and the FFT**, ending with a preview of QR factorization. The instructor organizes the entire course so far into two intertwined story lines and walks through them slowly.

Story Line 1: Analysis of $Ax = b$ (Existence and Uniqueness)

The first story line started with the problem $Ax = b$ and the observation that there are fundamental questions about **existence** and **uniqueness** of solutions. To address these analysis problems, the course adopted **vector space methods**, which generalize our two- and three-dimensional intuition to higher-dimensional spaces.

The chronological build-up was:

1. **Vector spaces and subspaces.** The course introduced abstract vector spaces and subspaces. The instructor recalled interesting examples such as the **subspace of zero-trace matrices** — examples meant to show that “vectors” need not be arrows; matrices can form vector spaces too.
2. **Span.** Given a set of vectors, the span is the minimum linear space (smallest subspace) that contains them.
3. **Linear independence.** Whether every vector in a set carries valuable information, or whether one of them can be generated as a linear combination of the others.
4. **Basis and dimension.** Given a vector space, find a minimum set of vectors whose span equals that space. The number of vectors in such a minimal spanning set is the **dimension** of the space.
5. **Geometric tools beyond linear structure.** On top of the vector-space skeleton, the course added geometric structure. So far only the **Euclidean norm** has been used as the standard norm — the instructor emphasizes that up to this point **no other way of measuring the size of a vector has been introduced** (this changes in the norm lectures coming up). The Euclidean norm squares the entries, sums them, and takes the square root: the Euclidean distance to the origin.
6. **Euclidean inner product.** This gives angle information. In two and three dimensions the inner product corresponds to the geometric angle; the course extended the angle notion to high dimensions using the **Cauchy-Schwarz inequality**, which says the inner product, normalized by the norms, always lies between -1 and $+1$. That bound is exactly what lets us keep calling it a cosine of an angle in high dimensions.
7. **Orthogonality.** Orthogonality of vectors, orthonormal sets.
8. **Hyperplanes and half-spaces.** Described as really important concepts, e.g., in machine learning — a single neuron is essentially a half-space classifier that tells you which half-space your data lies in.
9. **Four fundamental subspaces.** Moving into the analysis of $Ax = b$, the course built the four fundamental subspaces by viewing the matrix as a collection of column vectors versus a collection of row vectors: **column space** vs. **row space**, plus the spaces orthogonal to them (left null space and null space).

10. **Rank.** The instructor notes he didn't list it explicitly above, but **rank** — the common dimension of the column space and the row space (proved equal) — is central. The existence/uniqueness analysis depends on **two factors**: the **shape** of A (square, tall, fat) and the **rank** of A . Based on those, conclusions about existence and uniqueness follow.
-

Story Line 2: Solving $Ax = b$ via Factorizations

The second story line uses $Ax = b$ as an excuse to introduce a sequence of powerful concepts. The strategy:

- Identify which A matrices make $Ax = b$ **easy** to solve: **diagonal, triangular, and orthogonal (unitary)** matrices.
- The basic strategy is to convert a general $Ax = b$ into a **sequence of simple problems**, which is equivalent to writing A as a **product of simple matrices**.

Along this story line the course developed:

- **Basis change and similarity transformation** as a trick to convert a given matrix into diagonal form. This led to **eigenvalues and eigenvectors** and the concept of being **diagonalizable**. The course saw this trick may fail — some matrices cannot be represented by a diagonal matrix in any basis (not diagonalizable).
- **Complex vectors and projection operations**: projection of a vector onto another vector, and projection of a vector onto a subspace.
- **Special matrices**: diagonal, triangular, permutation, Hermitian, unitary, (orthogonal) projection matrices, Toeplitz, Hankel (in homework), and **circulant** matrices (also in homework, and revisited in detail today).
- **Normal matrices**, the big family. Within normal matrices: **Hermitian** and **unitary**; the normal family also partitions into unitary, Hermitian, **skew-Hermitian**, and others with mixed eigenvalue structure. A normal matrix is precisely a **unitarily diagonalizable** matrix. The search was for a diagonalizable T ; not all matrices are diagonalizable, but within the diagonalizable ones, the **unitarily** diagonalizable ones are the normal matrices, and their eigenvalue locations give the characteristic subsets:
 - eigenvalues on the unit circle $\rightarrow$ **unitary**;
 - eigenvalues real $\rightarrow$ **Hermitian**;
 - eigenvalues on the imaginary axis $\rightarrow$ **skew-Hermitian**.
- **Hermitian sub-classification by eigenvalue sign**. Hermitian matrices are useful for defining real-valued **quadratic functions** of complex vectors. If eigenvalues are all nonneg: **positive semidefinite**; all positive: **positive definite** (defines a **convex** quadratic); all negative: negative definite (defines a **concave** quadratic); mixed: **indefinite** (saddle structure).
- **Inertia and Sylvester's law**. Inertia is the sign pattern (counts of positive/negative/zero eigenvalues). **Star-congruence preserves inertia** (Sylvester's law of inertia): if two Hermitian matrices are star-congruent, they have the same inertia.
- **Matrix square root**. Because every PD matrix has inertia $(n, 0, 0)$, it is star-congruent to the identity, so any PD matrix can be written $A = SS^*$ with invertible S . There are infinitely many such square roots (multiply S by any unitary). Generic notation is used when the specific structure doesn't matter (e.g., random vector generation). Uses include solving PD systems via the lower triangular (Cholesky) square root and, in estimation theory, factoring the observation covariance to build **causal estimators (Wiener filters)** — the causal Wiener filter needs the lower triangular square root because lower triangular matrices are causal linear operators; the non-causal Wiener filter has no causality constraint.

Catalog of Factorizations Seen So Far

The instructor lists the factorizations encountered, all instances of “write a matrix as a product of simple matrices”:

- **PLU**: permutation $\times$ lower triangular $\times$ upper triangular.
 - **Eigenvalue decomposition**: $A = T\Lambda T^{-1}$ — T is not necessarily unitary in general, but for **normal** matrices T becomes a unitary (simple) matrix.
 - **Schur decomposition**: you may not be able to diagonalize a square matrix, but you can **triangularize** it using a unitary matrix: $A = UTU^*$. Still in the “product of simple matrices” form.
 - **Cholesky**: for PD matrices, the LU factorization specializes to Cholesky, a symmetric (lower triangular $\times$ its conjugate transpose) factorization.
 - **QR**: to be started next, with detail.
 - **SVD (Singular Value Decomposition)**: described as “the queen of factorizations” and “the best thing that happened to us.” It writes any matrix using **two** unitary matrices (unlike Schur, which uses one) and a **real, nonneg diagonal** matrix in between, and it applies to **non-square** matrices. Geometric reasoning will be given in later lectures.
-

Recap: Coloring and Whitening (with Student Q&A)

The instructor revisits last lecture’s random-vector application, since it ties matrix square roots to a concrete computation.

Before the construction, he recalls the meaning of a covariance matrix: it is a **library of pairwise covariances** for all pairs of random variables inside a vector. Correlation/covariance is being used to capture the **linear relationship** between two random variables. For a random vector, the covariance matrix collects all such pairwise relationships in one matrix.

Setup. `randn(n, L)` in MATLAB (with a Python counterpart) generates an $n \times L$ matrix that can be viewed as L independent samples of an n -dimensional **uncorrelated** Gaussian vector z with zero mean and identity covariance. By default you get **white** (uncorrelated) vectors.

Coloring. To produce a vector y with a desired covariance C_y , factor $C_y = SS^*$ (any square root — not restricted to PD or lower triangular) and set $y = Sz$.

Why correlation appears. Writing the coloring matrix as A , the first element $y_1 = A_{11}z_1 + A_{12}z_2 + \dots$, the second $y_2 = A_{21}z_1 + A_{22}z_2 + \dots$. Because the y_i are built from the **same** pool of underlying random variables $z_1, z_2, \dots$, it is logical to expect them to become correlated. The instructor stresses this is the intuition for *why* mixing creates correlation.

When mixing does NOT create correlation — student question. A student effectively asks which A keeps the output uncorrelated. Two cases: - If A is **diagonal**, each y_i depends only on z_i , so they remain uncorrelated (trivially). - But there are **non-diagonal** A that still leave y uncorrelated. Compute the correlation of the colored vector:

$$\mathbb{E}[yy^*] = \mathbb{E}[Azz^*A^*] = A\mathbb{E}[zz^*]A^* = AA^*.$$

For this to equal the identity we need $AA^* = I$, i.e., A **unitary**. **Therefore if you use a unitary coloring matrix you are not actually coloring** — despite y_1 and y_2 being built from the same sources, a unitary mixing keeps them uncorrelated. (A student also raised the idea of using a *random* matrix; the instructor set that aside as more complicated and chose to keep A fixed.)

Why the square root (student asked directly). A student said they didn’t understand why the square root is needed. The instructor re-derived it: since the correlation involves y **and** its conjugate transpose,

$$\mathbb{E}[yy^*] = \mathbb{E}[C_y^{1/2}zz^*C_y^{*/2}] = C_y^{1/2}I C_y^{*/2} = C_y.$$

The coloring matrix appears **twice** (once as itself, once conjugate-transposed). That is exactly why you need the *square root* of C_y and not C_y itself.

Final step. Add the desired mean. Up to this point y is zero-mean; you then shift by whatever mean you want.

A new homework covering these random-vector concepts will be assigned **after the midterm**.

LTI Systems from a Linear-Algebra Viewpoint

The instructor now reinterprets undergraduate **Signals and Systems** material through linear algebra, aiming to give intuition. (He notes he shared review notes for the EE 301-type material and recommends Oppenheim & Willsky's book — “a bit thick, but a very good book.”)

Linearity

An **LTI system** has two properties: **linearity** plus **time invariance**.

Linearity is the **superposition** property, which combines: - **Scaling (homogeneity)**: if you scale the input by α , the output scales by α . - **Additivity**: if input x_1 produces output y_1 and input x_2 produces y_2 , then $x_1 + x_2$ produces $y_1 + y_2$ (for a linear, not necessarily time-invariant, system).

Decomposition consequence. Suppose the input can be written as a weighted sum of special basis signals u_k :

$$x = \sum_{k=1}^m a_k u_k.$$

Then by linearity the output is the **same** weighted combination of the individual responses y_k :

$$y = \sum_{k=1}^m a_k y_k,$$

where y_k is the system's response to u_k . The instructor stresses the practical power: **run an experiment** feeding each basis signal u_k into the system and **record** its output y_k ; afterward you no longer need the system. As long as your basis signals are “rich” (any input of interest lies in their span), you can compute the output of any new input from the recorded y_k 's.

Time Invariance

Time invariance means: if you **shift the input in time, the output shifts by the same amount** and otherwise keeps the same shape. The system's behavior does not depend on *when* the input is applied.

Illustration. Apply an input (e.g., a pulse over samples 0 to 1) and get some output shape. Now apply the same input **delayed by 5 samples**. A time-invariant system produces the **same output shape, delayed by 5 samples**. The shape of input and output never changes; the system is not sensitive to the time of application. (Note: time invariance by itself can hold for linear *or* nonlinear systems.)

Finite-Length Signals and Circular Convolution

To stay within linear algebra (avoiding functional analysis), the lecture restricts to **finite-length signals** represented by N -dimensional vectors. (The full Signals and Systems course handles infinite-extent and continuous signals, which require vector-space/functional-analysis machinery.) The instructor says students already saw the Fourier-transform idea in homework; here he wants to give more intuition and formalize why taking the Fourier transform helps.

With input and output both constrained to length N , LTI systems are represented by **circular convolution**. Using r as the sample index to avoid overloading the length symbol,

$$y[r] = \sum_{k=0}^{N-1} h[k] x[(r - k) \bmod N].$$

The $\bmod N$ (modulo- N) operation always returns an index in $\{0, 1, \dots, N - 1\}$, which makes the shift **circular** (a rotation): entries that “fall off” one end **wrap around** to the other end.

The Circular Shift Operator Z

The instructor demonstrates the circular shift on a concrete 4-vector. Starting from

$$h = \begin{bmatrix} 2 \\ -1 \\ 1 \\ -2 \end{bmatrix},$$

a one-step circular (downward) shift moves every entry down by one and the **bottom entry wraps to the top**. This rotation is itself a **linear operation**, represented by a matrix Z that is a **rotated/shuffled identity matrix**:

$$Z = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

For the displayed vector this gives

$$Zh = \begin{bmatrix} -2 \\ 2 \\ -1 \\ 1 \end{bmatrix}.$$

Reading off the rows: row 2 of Z picks row 1’s entry, row 3 picks row 2’s, etc., and the top row picks the bottom entry — exactly the wrap-around. Thus a one-step circular shift is Zh ; a k -step circular shift is $Z^k h$ (because $Z^2 h = Z(Zh)$, and so on). Z is a **permutation matrix**.

Circular Convolution as a Circulant Matrix

Circular convolution is a **weighted sum of circularly shifted copies** of h , weighted by the input entries:

$$y = x[0] h + x[1] Zh + x[2] Z^2 h + \dots + x[N - 1] Z^{N-1} h = \left(\sum_{m=0}^{N-1} x[m] Z^m \right) h.$$

The index of each x entry tells you **how much shift** to apply to h : $x[0]$ multiplies the un-shifted h , $x[1]$ the one-shift Zh , $x[2]$ the two-shift $Z^2 h$, up to $x[N - 1]$ multiplying $Z^{N-1} h$. Equivalently, collecting these shifted copies of h as columns produces the **circulant matrix** H_h , so the output is a matrix-vector product:

$$y = H_h x, \quad H_h = [h \quad Zh \quad Z^2 h \quad \dots \quad Z^{N-1} h].$$

For the downward-shift convention above, the same matrix can be written as a polynomial in the shift matrix,

$$H_h = \sum_{m=0}^{N-1} h[m] Z^m = \begin{bmatrix} h_0 & h_{N-1} & h_{N-2} & \dots & h_1 \\ h_1 & h_0 & h_{N-1} & \dots & h_2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ h_{N-1} & h_{N-2} & h_{N-3} & \dots & h_0 \end{bmatrix}.$$

Because $y = H_h x$, this is a **linear system**; its special time-invariance (with respect to *circular* shifts) forces the **circulant** structure on H_h . Circular convolution is commutative, so the expression $(\sum_m x[m] Z^m) h$ gives the same vector while emphasizing the instructor’s “weighted shifted copies of h ” viewpoint.

Multiplication Count for a General Linear System

How many multiplications does computing $y = Hx$ require for a **general** (non-circulant) $N \times N$ matrix? Each output entry y_i needs N multiplications, and there are N entries, so N^2 **multiplications** (additions are cheaper; multiplications are the costly operations counted). A general, unstructured linear-system output costs N^2 . The point of the rest of the lecture is to **exploit the circulant structure** to do far better — and that exploitation is tied to the Fourier transform.

Complex Exponentials Are Eigenvectors of Circulant Matrices

Consider a **structured input**: a complex exponential. Define the family of vectors f_k (indexed by k) whose r -th entry is

$$(f_k)[r] = e^{j2\pi kr/N}, \quad r = 0, \dots, N-1.$$

where k/N is the **frequency** of the complex exponential. So f_k is the “state vector” at frequency k/N : its entries are $1, e^{j2\pi k/N}, e^{j2\pi k \cdot 2/N}, \dots, e^{j2\pi k(N-1)/N}$.

Key fact. When you pass f_k through the circulant system, you get the **same vector back, scaled by a constant** that depends only on the frequency:

$$Hf_k = \hat{h}_k f_k.$$

Therefore each f_k is an **eigenvector** of H , with eigenvalue $\hat{h}_k$. The instructor frames the discovery as a question to the class — “what would you call this?” — answer: an eigenvector, since the input comes back in the same direction, merely scaled.

The eigenvalue is the DFT of h . The scaling constant is the **inner product** of the impulse response h with the complex exponential vector:

$$\hat{h}_k = \sum_{r=0}^{N-1} h[r] e^{-j2\pi kr/N},$$

i.e., the Fourier transform of h evaluated at frequency k/N under the forward-DFT convention clarified later in the lecture. The minus sign appears because the DFT is F^*h when F is written with positive-exponent columns.

The shift-matrix proof is the cleanest way to see the scaling. Since

$$Zf_k = e^{-j2\pi k/N} f_k,$$

we get

$$H_h f_k = \left(\sum_{m=0}^{N-1} h[m] Z^m \right) f_k = \left(\sum_{m=0}^{N-1} h[m] e^{-j2\pi km/N} \right) f_k.$$

Thus the complex exponential vector comes back unchanged in direction and scaled by the DFT coefficient of h . The instructor does not grind through every row multiplication, but he says this can be verified from the shift structure and connects it to what students already saw in the homework.

Diagonalizing a Circulant Matrix: the DFT Matrix

Stacking the eigenvector equations for all frequencies $k = 0, \dots, N-1$ as columns:

$$H [f_0 \ f_1 \ \cdots \ f_{N-1}] = [f_0 \ f_1 \ \cdots \ f_{N-1}] \Lambda,$$

where multiplying the matrix of eigenvectors **from the right** by a diagonal matrix Λ scales each column by its eigenvalue. Calling the matrix of complex-exponential columns the **Fourier matrix** F (columns f_k) and $\Lambda = \text{diag}(\hat{h}_0, \dots, \hat{h}_{N-1})$:

$$HF = F\Lambda \implies H = F\Lambda F^{-1}.$$

The eigenvalues on the diagonal of Λ are the **DFT of** h evaluated at the N frequencies.

F Is Orthogonal but Not Orthonormal

The columns of F are **orthogonal** to each other. The instructor asks what that says about F^{-1} ; the answer “conjugate transpose” is *almost* right — but you must include a factor of $1/N$ because the columns are orthogonal but **not orthonormal**.

Compute F^*F : a typical entry is the inner product of one complex-exponential column with another. For example, in F^* a row is $1, e^{-j2\pi/N}, \dots, e^{-j2\pi(N-1)/N}$ and in F a column is $1, e^{j2\pi/N}, \dots$. The diagonal inner product gives: $1 \cdot 1 = 1$, then $e^{-j2\pi/N}e^{j2\pi/N} = 1$ (conjugate pairs each give 1), and so on — summing N ones gives N . Off-diagonal inner products sum to 0. Hence

$$F^*F = FF^* = NI.$$

So F is a **scaled unitary** matrix, and

$$F^{-1} = \frac{1}{N}F^*.$$

(The instructor notes the 4-dimensional version of this orthogonality was already verified in homework.)

The Normalized (Unitary) DFT Matrix

The squared norm of each column of F is N , so each column has norm $\sqrt{N}$. Dividing F by $\sqrt{N}$ gives a genuinely **unitary** matrix:

$$\tilde{F} = \frac{1}{\sqrt{N}}F, \quad \tilde{F}^*\tilde{F} = I.$$

Then the circulant matrix diagonalizes as

$$H = \tilde{F}\Lambda\tilde{F}^*,$$

a unitary $\times$ diagonal $\times$ unitary-conjugate decomposition. **Therefore every circulant matrix is unitarily diagonalizable — it is a normal matrix.** But its eigenvalues (the DFT values $\hat{h}_k$) are **not** restricted to the unit circle or the real line; they sit wherever the Fourier transform of h lands. So **circulant (circular convolution) matrices are a special case of normal matrices.**

Why This Matters: the FFT and Fast Algorithms

A Historical Aside (Cold War)

The instructor recommends a Veritasium episode on how the fast computation of the Fourier transform emerged during the **Cold War**. The U.S. wanted to detect whether the Soviets were conducting **underground nuclear tests**, so sensors were placed and the measurements analyzed for characteristic frequencies. Fast Fourier computation was critical, and **Tukey** — who co-developed the fast procedure — was on the U.S. scientific committee involved in this detection effort. (Cooley–Tukey FFT.)

Operation Count Without the FFT

Computing $y = Hx$ directly costs N^2 multiplications. Using the factorization $H = \tilde{F}\Lambda\tilde{F}^*$ (i.e., $H = \frac{1}{N}F\Lambda F^*$) means: 1. Compute the frequency response/eigenvalues $\hat{h}_k$, i.e. the DFT of h : naively N^2 multiplications if done as dense Fourier-matrix multiplication. 2. Multiply x by the Fourier matrix (take the **DFT** of x): naively N^2 multiplications (an $N \times N$ matrix times a vector). Each output entry is an inner product of x

with a complex exponential — i.e., the DFT. 3. Multiply by the **diagonal** Λ : only N multiplications. This is the step that **converts convolution into elementwise multiplication** — the values being multiplied are the coordinates of x in the Fourier (orthogonal) basis, and the diagonal entries are the output's coordinates. 4. Multiply by $\frac{1}{N}F$ (the **inverse DFT**) to return to the original coordinates: another N^2 .

So done naively, the transform route costs about $3N^2 + N$ if $\hat{h}$ is not already available — **more** expensive than the direct N^2 multiply. By itself, changing to the Fourier basis is not a win.

Operation Count With the FFT

The payoff is that the multiplication by F (or F^*) need **not** be done as a dense N^2 matrix-vector product. Tukey's **FFT** exploits the special structure of the Fourier matrix (the **butterfly structure**) to compute the DFT in $O(N \log N)$ instead of $O(N^2)$. The instructor does not derive the butterfly but emphasizes the scale of the saving:

- DFT of h to get $\hat{h}_k$: $N \log N$ (or precompute once if the system is fixed).
- Forward DFT of x : $N \log N$.
- Elementwise/diagonal multiply: N (negligible at this scale).
- Inverse DFT: $N \log N$.

Total $\approx 3N \log N$ (plus negligible terms) for one convolution including the transform of h , or about $2N \log N$ if $\hat{h}$ was precomputed. This replaces N^2 . For $N = 1000$: direct is $\sim$ one million operations; the FFT route is on the order of $1000 \times \log(1000)$ — a few thousand. “A huge saving,” especially in the **big-data era** where N can be in the millions.

The Inverse DFT and the $1/N$ Factor

The $1/N$ in the inverse DFT comes directly from $F^{-1} = \frac{1}{N}F^*$. The inverse transform is

$$x[r] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j2\pi kr/N}.$$

This is why the inverse Fourier transform carries the $1/N$ factor and the forward one does not.

Displacement Structure (Generalization)

Circulant matrices are one example of structured matrices that admit fast multiplication. The instructor mentions the broader concept of **displacement structure** (there is a book on it): for various structured matrices, you try to convert the N^2 matrix multiplication into a faster $O(N \log N)$ (or other) form by exploiting structure. These are the **fast algorithms**. Regular (unstructured) matrix multiplication needs N^2 ; structured matrices like circulants can be sped up dramatically.

DFT Conventions and Sign of Exponents (Student Q&A)

A student asked about the DFT formula and the placement of minus signs. The instructor clarifies the bookkeeping:

- Writing F^*x gives the **DFT** (forward); writing $\frac{1}{N}F X$ gives the **inverse DFT**. Since F is not unitary, its inverse is $\frac{1}{N}F^*$, not just F^* .
- In the F matrix as written, the entries carry **no minus sign** in the exponent. In F^* (conjugate), the columns become rows and the exponents pick up **minus signs**. So when you multiply F^*x you get terms $\sum_{r=0}^{N-1} x[r]e^{-j2\pi kr/N}$ — the familiar forward DFT with the **minus sign**. The Fx version has $e^{+j2\pi kr/N}$.
- The summation $1/N$ factor in the inverse appears because F is **scaled unitary**, not unitary.

Symmetric (Normalized) Convention

If instead you use the **normalized** basis vectors (dividing each by $\sqrt{N}$), then the matrix is unitary and its conjugate is its inverse, making the forward and inverse transforms **symmetric** in their scaling:

$$\tilde{X}[k] = \frac{1}{\sqrt{N}} \sum_{r=0}^{N-1} x[r] e^{-j2\pi kr/N}, \quad x[r] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \tilde{X}[k] e^{+j2\pi kr/N}.$$

The instructor calls $\tilde{X}$ the **normalized Fourier transform**. The choice is “not a big deal,” but it explains why the asymmetric convention has a $1/N$ in the inverse and nothing in the forward.

Preview: QR Factorization

The instructor closes with one sentence on what comes next. **QR factorization** writes a matrix as a **unitary matrix times an upper triangular matrix**:

$$A = QR,$$

with Q unitary and R upper triangular — both simple matrices. He muses that he doesn’t know why it’s called “Q” (perhaps Q for unitary?) and jokes that maybe it should be called “**UT factorization**” (unitary–triangular). The claim, to be proved next: **any square matrix can be written as a unitary matrix times an upper triangular matrix**, and there are different ways to obtain it. “We will meet in the evening. See you tonight.”

Instructor Remarks and Study Guidance

- The course is two intertwined story lines: **analyzing** $Ax = b$ (vector spaces, four fundamental subspaces, existence/uniqueness from shape + rank) and **solving** $Ax = b$ (factor A into simple matrices).
- All factorizations (PLU, eigendecomposition, Schur, Cholesky, QR, SVD) are instances of “write A as a product of simple matrices.” SVD is the “queen” — two unitaries and a real nonneg diagonal, works for any matrix.
- A covariance matrix is a library of pairwise covariance/correlation information; coloring starts from white Gaussian samples and uses a square root of the target covariance.
- **A unitary coloring matrix does no coloring:** $AA^* = I$ keeps the output white. The square root appears because the covariance computation puts the coloring matrix on both sides ($\mathbb{E}[yy^*] = SS^*$).
- **Complex exponentials are eigenvectors of circulant matrices;** the eigenvalues are the DFT of the impulse response. Hence circulant matrices are normal and unitarily diagonalized by the (normalized) DFT matrix.
- The (unnormalized) Fourier matrix satisfies $F^*F = NI$: scaled unitary, so $F^{-1} = \frac{1}{N}F^*$. This is the origin of the $1/N$ in the inverse DFT.
- The FFT reduces the DFT cost from N^2 to $O(N \log N)$ via the butterfly structure; including the transform of h , the circular convolution workflow is about $3N \log N$, or about $2N \log N$ if the frequency response is precomputed.
- A new random-vector homework comes after the midterm; reread the EE 301 / Oppenheim–Willsky background if Signals and Systems is rusty.

Source and Coverage Note

Source: `corrected/lecture16_corrected.md`.

Coverage: Opening reflection; full chronological mid-course review of both story lines (vector spaces, span, linear independence, basis/dimension, Euclidean norm and inner product, Cauchy-Schwarz, orthogonality, hyperplanes/half-spaces, four fundamental subspaces, rank, existence/uniqueness via shape + rank; easy $Ax =$

b cases, factorization strategy, basis change/similarity, eigenvalues, diagonalizability, special matrices, normal matrices and their subclasses, Hermitian sign classes, inertia, Sylvester's law, matrix square roots, catalog of factorizations including SVD preview); covariance as a library of pairwise relationships; coloring/whitening recap with the unitary-matrix and square-root student Q&A; LTI systems (linearity/superposition, basis-signal decomposition, time-invariance illustration); finite-length signals and circular convolution; the circular shift operator Z as a permutation matrix with the worked 4-vector example and shifted vector; circulant matrix $H_h = [h \ Zh \ \cdots \ Z^{N-1}h] = \sum h[m]Z^m$, its explicit row structure, and the N^2 multiplication count; complex exponentials as eigenvectors with eigenvalue = forward DFT of h , including the Zf_k proof step; diagonalization $H = F\Lambda F^{-1}$; $F^*F = NI$, scaled-unitary nature, normalized unitary DFT matrix, circulant matrices as normal matrices; FFT motivation (Cold War / Tukey), operation-count comparison N^2 vs $O(N \log N)$, including the DFT of h , displacement structure; DFT sign/convention student Q&A and the symmetric normalized convention; QR preview.